

Assignment: Autoencoders on Tabular Data

Unsupervised Learning

Mode: Individual or pairs
Submission: Jupyter notebook (.ipynb)

In the lecture you saw autoencoders on images. Today you will apply the same ideas to **tabular data** — numbers, not pixels. The core logic is identical: compress through a bottleneck, reconstruct, and use the reconstruction error and the latent space to learn something about the data.

1 Setup and Data

You will use the **Wine Quality** dataset (red wine), available from scikit-learn or directly from UCI:

```
import pandas as pd
url = ("https://archive.ics.uci.edu/ml/"
      "machine-learning-databases/wine-quality/winequality-red.csv")
wine = pd.read_csv(url, sep=';')
```

Each row is a wine described by 11 chemical properties (fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol). There is also a **quality** column (score from 3 to 8) — **set it aside**, do not use it as a feature. You will use it only at the end to interpret your results.

1. Load the data and standardise the 11 features (zero mean, unit variance).
2. Report the shape and check for missing values.

2 PCA Baseline

- 2.1 Apply PCA and plot the cumulative explained variance. How many components capture 90% of the variance?
- 2.2 Reconstruct the data using only 2 components. Compute the MSE for each wine. Plot the distribution of reconstruction errors.
- 2.3 Colour the 2D PCA scatter plot by **quality**. Do high-quality and low-quality wines separate?

Identify the 10 wines with the highest PCA reconstruction error. What quality scores do they have? Are the “hard to reconstruct” wines also unusual wines?

3 Autoencoder

Build a simple autoencoder in PyTorch:

Encoder: 11 -> 8 -> 4 -> d
Decoder: d -> 4 -> 8 -> 11

Use ReLU activations, MSE loss, and Adam optimiser. Try $d = 2$ and $d = 4$.

- 3.1 Train for 200 epochs. Plot the loss curve. Compare the final MSE with the PCA reconstruction error at the same d .
- 3.2 Extract the 2D latent vectors ($d = 2$) and scatter plot them coloured by quality. Compare with the PCA 2D plot.
- 3.3 **Nearest neighbours.** Pick 3 wines you like from the dataset (look at the features — high alcohol? low acidity? your call). Find their 5 nearest neighbours in the autoencoder latent space vs in the original 11D space. Do the neighbours change?

Does the autoencoder latent space separate quality levels better than PCA? If yes, what non-linear structure might it be capturing? If no, why not?

Look at the neighbours of your chosen wines. Does the latent space group wines by similar chemical profiles, or by something else?

4 Anomaly Detection via Reconstruction Error

The key idea: if a wine is “normal”, the autoencoder reconstructs it well (low error). If it is unusual, the reconstruction is poor (high error).

- 4.1 Using your trained autoencoder ($d = 4$), compute the reconstruction error for every wine. Plot the distribution.
- 4.2 Flag the top 5% highest-error wines as anomalies. How many are there?
- 4.3 Compare with Isolation Forest on the same data. How many wines are flagged by **both** methods? Show them in a table with their features and quality score.
- 4.4 Create a scatter plot (PCA 2D or autoencoder 2D) marking normal wines, AE-only anomalies, Isolation Forest-only anomalies, and wines flagged by both.

Look at the wines flagged by both methods. What makes them unusual — is it one extreme feature, or a rare combination? Check their actual values.

Do the anomalies tend to be very bad wines (quality 3–4) or very good wines (quality 7–8), or both? What does this tell you about what “unusual” means in this context?

Aim to finish Sections 1–4 in the two hours. The analysis matters more than perfect code: if a plot looks interesting, write a sentence explaining why.